

Detecting and Characterizing Representation Drift in Multimodal Vision-Language Models

Sahil Mhatre
sm2997@cornell.edu
Cornell University
USA

Ali Hasan
ah2434@cornell.edu
Cornell University
USA

Corey Chen
cc2949@cornell.edu
Cornell University
USA

Abstract

Fine-tuning large multimodal models such as CLIP on narrow domains can induce “catastrophic forgetting,” where performance on general visual tasks degrades even as the target task improves. In this work, we study *representation drift*, defined as geometric changes in the image embedding space that arise during parameter-efficient fine-tuning with LoRA. Concretely, we fine-tune CLIP ViT-B/32 on the Food101 dataset using LoRA adapters on the vision transformer while treating CIFAR-10 as a held-out proxy for general knowledge, and we periodically save embeddings for both datasets throughout training.¹

Using these saved embeddings, we compute prototype-based classification accuracy together with two geometric drift metrics: cosine centroid displacement and Fréchet distance between the baseline and fine-tuned embedding distributions. The resulting stability–plasticity curves show that gains on Food101 are accompanied by a measurable drop in CIFAR-10 performance, and that both cosine drift and Fréchet distance exhibit a consistent increasing trend over training steps. We also perform a simple early-warning analysis by fitting a linear predictor from an intermediate checkpoint’s drift value to the final drift, and we quantify the resulting prediction error. Taken together, these experiments suggest that representation drift is structured and trackable, and that geometric monitoring can provide useful signals about model degradation during fine-tuning, even when labeled data for all out-of-domain capabilities are not available.

1 Introduction

The creation of Multimodal Vision–Language Models (VLMs) such as CLIP (Contrastive Language–Image Pre-training) has significantly reshaped the landscape of computer vision. By aligning visual and textual representations in a shared high-dimensional space, these models exhibit strong zero-shot capabilities, allowing them to classify images into categories that were never seen with explicit supervised labels. This generalization behavior emerges from training on massive, heterogeneous datasets collected from the web.

Despite their breadth, general-purpose models often fall short in domains that demand fine-grained or safety-critical decisions. A model trained on internet-scale data may recognize a generic “car,” whereas an autonomous driving system must reliably distinguish a “police cruiser” from a “taxi” or a “school bus.” In medical imaging, a general VLM might identify an “x-ray,” but a deployed clinical assistant needs to detect subtle markers of disease. To close this

gap between generality and specialization, practitioners routinely fine-tune foundation models on domain-specific datasets.

Fine-tuning typically improves performance on the new target task, a property referred to as *plasticity*. At the same time, it can erode performance on previously mastered tasks, which reflects a loss of *stability*. This degradation of prior knowledge, commonly described as catastrophic forgetting, poses a practical risk in long-lived systems. A VLM adapted to recognize food dishes may gradually lose accuracy on common objects, animals, or vehicles that were handled well in its original configuration. In many production settings, this failure mode is difficult to detect, because standard MLOps pipelines monitor validation loss only on the target dataset (for example, Food101), which continues to improve even as performance on out-of-domain data (for example, CIFAR-10) declines.

This project investigates the mechanism of such forgetting through the lens of representation drift. We use the term representation drift to describe geometric changes in the embedding space, including both displacement and distortion, that occur as model parameters are updated during fine-tuning. Our hypothesis is that forgetting isn’t random. It’s a direct trade-off where the model shifts its internal map to fit the new task, breaking the old map in the process.

Concretely, we study representation drift in CLIP ViT-B/32 when it is adapted to the Food101 dataset using Low-Rank Adaptation (LoRA) applied only to the vision transformer. CIFAR-10 is treated as a held-out proxy for general visual knowledge, and we periodically record embeddings for both datasets throughout training. This setup yields a controlled environment in which learning on the food domain is encouraged, while performance on the general domain is passively monitored but never reinforced.

Our contributions are threefold:

- (1) **Controlled Induction Framework:** We implement an end-to-end experimental pipeline that fine-tunes CLIP with LoRA adapters on the vision tower while freezing the rest of the model. This design allows us to induce drift in a controlled way and to attribute changes primarily to attention-layer re-weighting.
- (2) **Geometric Quantification Strategy:** We define and compute a dual set of metrics for monitoring drift over time: cosine-based centroid displacement to capture shifts in the global embedding center, and Fréchet distance between baseline and fine-tuned embedding distributions to capture higher-order distortion.
- (3) **Drift–Performance Coupling Analysis:** We jointly track drift metrics and classification accuracy on both Food101 and CIFAR-10 across training checkpoints, and we perform a simple early-warning analysis that uses an intermediate drift measurement to predict final drift. This analysis provides

¹All implementation details, including hyperparameters and data splits, are provided in Section ??.

initial evidence that geometric drift can serve as a useful indicator of emerging performance degradation on held-out domains.

2 Related Work

2.1 Vision–Language Pre-training

Radford et al. (2021) introduced CLIP, which established the now-standard paradigm of jointly training image and text encoders with a symmetric contrastive loss over paired data scraped from the web. This approach showed that natural language supervision can provide a flexible label space and support strong zero-shot transfer to downstream image classification problems. Subsequent work, including ALIGN and BLIP-2, scaled this recipe in terms of both data and model size and extended it to richer multimodal tasks such as captioning and visual question answering.

Our study builds directly on the CLIP architecture, and in particular the ViT-B/32 variant that remains widely used as a backbone for transfer learning and evaluation benchmarks. In contrast to prior efforts that mostly focus on improving pre-training or designing new multimodal objectives, we treat CLIP as a fixed starting point and ask how its learned representations change when it is adapted to a narrower domain using parameter-efficient fine-tuning. This perspective positions our work as a diagnostic layer on top of existing vision–language pre-training methods rather than as a competing pre-training objective.

2.2 Catastrophic Forgetting and Stability

The tension between learning new tasks (plasticity) and retaining prior knowledge (stability) is a fundamental challenge for neural networks. This often results in a phenomenon called “catastrophic forgetting” where neural networks abruptly forget previously learned information when trained on new data.

Recent literature focuses on mitigation of this effect. Wortsman et al. (2022) proposed “Wise-FT,” which demonstrated that the robustness of the zero-shot model can be recovered by linearly interpolating the weights of a fine-tuned model. Another approach to address this effect is Elastic Weight Consolidation, which penalizes changes to critical parameters.

While Worstman et al. characterize the outcome of forgetting (via accuracy drops on distribution shifts), there is limited work on documenting the process of forgetting geometrically during training. Our work aims to address this gap. Instead of proposing a raw mitigation strategy, we employ Krizhevsky (2009)’s CIFAR-10 dataset as an out-of-distribution “probe” to continuously monitor the topology of the embedding manifold. Our hypothesis is that forgetting is not random noise, but rather a structured geometric warping that can be detected via Fréchet Distance before accuracy metrics degrade.

2.3 Parameter-Efficient Fine-Tuning (PEFT) and Representation Drift

As multimodal models continue to grow, full-parameter fine-tuning becomes increasingly expensive and can lead to overfitting on relatively small domain-specific datasets. Parameter-efficient fine-tuning (PEFT) methods address this challenge by introducing a

small number of trainable parameters while keeping the majority of the pre-trained network frozen. Hu et al. (2021) proposed LoRA (Low-Rank Adaptation), which injects low-rank matrices into existing weight matrices and learns only these adapters, often reducing the number of trainable parameters by several orders of magnitude.

LoRA and related PEFT techniques have been widely adopted in both language and vision–language models, but their impact on representation drift and catastrophic forgetting is not yet fully understood. Prior work typically evaluates PEFT primarily in terms of target-task accuracy or robustness, with less emphasis on how the underlying embedding geometry shifts relative to the original model. Our experiments contribute to this gap by applying LoRA to the vision tower of CLIP ViT-B/32 and then explicitly measuring geometric drift in the image embedding space while monitoring accuracy on both the fine-tuning domain (Food101) and a held-out domain (CIFAR-10). The observed drift and associated forgetting indicate that even strongly constrained updates can materially alter the representation manifold, which motivates the need for systematic monitoring tools in PEFT pipelines.

3 Data

To simulate a realistic specialist–versus–generalist conflict, we adopt a dual-dataset experimental design. In this setup, one dataset drives the fine-tuning updates and encourages specialization, while a second dataset is held out from training and serves as a proxy for general visual knowledge. This design allows us to track how adapting CLIP to a new domain affects both in-domain performance and out-of-domain behavior over the course of training.

3.1 The Learn Set (Target Domain): Food101

We use the **Food101** dataset (Bossard et al., 2014) as the target domain for fine-tuning.

- **Description:** Food101 contains 101,000 images organized into 101 classes of food dishes (for example, *sushi*, *hamburger*, *pad thai*). The images exhibit substantial variation in view-point, lighting, plating, and background clutter.
- **Role in our study:** Food101 functions as the specialization driver. By training on this dataset, we encourage the model to emphasize features related to texture, color, and fine-grained appearance of food, which may come at the expense of more generic object features.
- **Usage in our pipeline:** In the notebook, we load the full Food101 training split and then reserve 1,000 images as a held-out Food101 evaluation set. The remaining 74,750 training images are used as the optimization dataset for LoRA fine-tuning across all epochs.

3.2 The Forget Set (Control Domain): CIFAR-10

We treat **CIFAR-10** (Krizhevsky, 2009) as a compact proxy for general visual knowledge.

- **Description:** CIFAR-10 consists of 60,000 low-resolution images (32×32 pixels) spread across 10 mutually exclusive classes: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, *truck*. The classes cover a broad set of everyday objects and animals.

- **Role in our study:** These classes are distinct from food categories, so performance changes on CIFAR-10 are indicative of how much the model’s general object recognition ability is affected by specializing on Food101. A drop in CIFAR-10 accuracy suggests that features relevant to generic objects (for example, wheels, wings, or fur) are being overwritten or de-emphasized.
- **Usage in our pipeline:** CIFAR-10 is strictly held out from the optimization loop. We load the CIFAR-10 test split, randomly shuffle it, and select a fixed subset of 2,000 images that serve as our evaluation set for all checkpoints. At each saved checkpoint, we compute embeddings and prototype-based accuracy on this fixed subset but never backpropagate through CIFAR-10 examples.

3.3 Preprocessing and Prompt Construction

All image and text inputs are preprocessed using the standard CLIP utilities corresponding to ViT-B/32.

- **Image preprocessing:** We rely on the CLIPProcessor from the Hugging Face transformers library, which applies CLIP’s default resizing, center-cropping, and normalization steps before feeding images to the vision encoder. This ensures that both Food101 and CIFAR-10 images are mapped into the same embedding space under the pre-trained CLIP configuration.
- **Text prompting:** For both datasets, we construct simple class-level prompts of the form “a photo of a {label}”. These prompts are tokenized with the CLIP tokenizer to produce text embeddings for each class. During evaluation, we use these text embeddings as prototypes and classify each image by selecting the class whose prototype has the highest cosine similarity with the image embedding.
- **Consistency across checkpoints:** The same preprocessing pipeline, prompt templates, and evaluation subsets are reused at every checkpoint. This consistency is crucial, because it allows us to attribute changes in performance and geometry to the evolving model parameters rather than to changes in the data pipeline.

4 Methods

4.1 Theoretical Framework: Contrastive Fine-Tuning

CLIP is trained with a symmetric contrastive objective over batches of paired images and texts. Let I_i and T_i denote the ℓ_2 -normalized embeddings of the i -th image and text in a batch of size N . The model constructs a similarity matrix $S_{ij} = I_i \cdot T_j$ and optimizes a cross-entropy loss that encourages high similarity for matched pairs ($i = j$) and low similarity for mismatched pairs ($i \neq j$). In practice, this loss is applied in both directions (image-to-text and text-to-image), averaged across directions, and scaled by a fixed temperature, following the standard CLIP formulation.

During fine-tuning on Food101, the gradients of this contrastive loss update the model parameters $\mathbf{W}$ to better align food images with their corresponding text prompts. Our working hypothesis is that these updates move the parameters from the pre-trained

checkpoint $\mathbf{W}_0$ to $\mathbf{W}_t$ in directions that improve alignment on the food manifold, while simultaneously distorting regions of the embedding space that support general object recognition as measured on CIFAR-10.

4.2 Low-Rank Adaptation (LoRA)

Instead of full fine-tuning, we employ Low-Rank Adaptation (LoRA) to constrain the update space. For a weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA parameterizes the update as

$$W = W_0 + \frac{\alpha}{r} BA, \quad (1)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank matrices that are learned during fine-tuning. In the standard configuration, B is initialized to zero and A is initialized from a small Gaussian distribution, so that the effective weight initially equals W_0 .

In our setup, we attach LoRA adapters only to the q_proj and v_proj projection layers in the attention blocks of the CLIP ViT-B/32 vision transformer. Attention layers control how spatial features are routed and combined, so modifying them effectively steers which visual patterns are emphasized. We set the LoRA rank to $r = 8$ and the scaling factor to $\alpha = 16$, with a LoRA dropout of 0.1, yielding 294,912 trainable parameters out of 87,750,912 (approximately 0.34% of the model). All other CLIP parameters, including the text encoder, remain frozen during training.

4.3 Drift Quantification Metrics

To quantify representation drift during fine-tuning without relying only on labeled accuracy, we compute geometric metrics at each checkpoint t using the saved image embeddings. All embeddings are ℓ_2 -normalized before analysis, which makes cosine-based quantities easier to interpret.

4.3.1 Cosine Drift (Global Centroid Displacement). Cosine drift measures how far the global mean embedding for a dataset moves from its baseline position. Let μ_t denote the mean of the image embeddings for a given dataset (for example, CIFAR-10) at checkpoint t , and let μ_0 be the corresponding mean at the baseline (step 0). The cosine drift is defined as

$$D_{\cos}(t) = 1 - \frac{\mu_t \cdot \mu_0}{\|\mu_t\| \|\mu_0\|}. \quad (2)$$

Values of $D_{\cos}(t)$ close to 0 indicate that the global embedding center is stable relative to the baseline, whereas larger values reflect substantial angular shifts. We compute this metric separately for the CIFAR-10 evaluation set and the Food101 evaluation set at each checkpoint, and focus on CIFAR-10 drift when analyzing forgetting.

4.3.2 Fréchet Distance (FD). While cosine drift captures movement of the mean, it does not describe changes in the spread or shape of the embedding distribution. To capture higher-order geometric effects, we use a Fréchet Distance (FD) metric analogous to the Fréchet Inception Distance used in generative modeling. Let (μ_0, Σ_0) and (μ_t, Σ_t) denote the empirical mean and covariance of the image embeddings at baseline and at checkpoint t , respectively. The Fréchet Distance is given by

$$FD(t) = \|\mu_t - \mu_0\|_2^2 + \text{Tr} \left(\Sigma_t + \Sigma_0 - 2(\Sigma_t \Sigma_0)^{1/2} \right). \quad (3)$$

Increases in $FD(t)$ indicate that the embedding manifold has shifted, changed shape, or both, even if the global mean remains relatively stable. As with cosine drift, we compute FD for both CIFAR-10 and Food101 and use the CIFAR-10 FD curve to study how general-domain geometry evolves during specialization.

4.4 Experimental Protocol

We run a controlled fine-tuning experiment using the hyperparameters summarized in Table 1. The CLIP backbone is initialized from the `openai/clip-vit-base-patch32` checkpoint, and only the LoRA adapters in the vision tower receive gradient updates.

Table 1: Hyperparameters for Controlled Fine-Tuning

Parameter	Value
Base Model	openai/clip-vit-base-patch32
Optimizer	AdamW
Learning Rate	5×10^{-5}
Weight Decay	0.01
Batch Size	32
Epochs	5
Checkpoint Frequency	Every 200 steps
LoRA Rank (r)	8
LoRA Alpha (α)	16
LoRA Dropout	0.1

Dual-Loop Analysis. To keep analysis computationally feasible while still tracking drift over time, we decouple training from metric computation. The procedure consists of two stages:

- (1) **Training Loop.** We train the LoRA-augmented CLIP model on the Food101 training subset for 5 epochs. At step 0 (before any updates), we extract and save baseline embeddings for the CIFAR-10 and Food101 evaluation sets. During training, every 200 optimization steps we run batched inference on the fixed CIFAR-10 and Food101 evaluation sets, extract their image embeddings, and store them as compressed .npz files associated with the current global step.
- (2) **Analysis Loop.** After training completes, we iterate over all recorded checkpoints. For each checkpoint, we load the saved embeddings for CIFAR-10 and Food101, compute prototype-based accuracies using the class text prompts, and evaluate cosine drift and Fréchet Distance relative to the baseline embeddings. The resulting metrics are aggregated into a single `final_merged_metrics.csv` file, which forms the basis for the stability-plasticity plots and geometric drift visualizations in the Experiments section.

5 Experiments and Results

5.1 The Stability-Plasticity Dilemma

Our first experiment quantifies the trade-off between learning the new task and retaining prior knowledge. Figure 1 shows classification accuracy on Food101 and CIFAR-10 across training checkpoints.

At step 0, the zero-shot CLIP model starts with relatively strong CIFAR-10 accuracy and moderate Food101 accuracy. As training

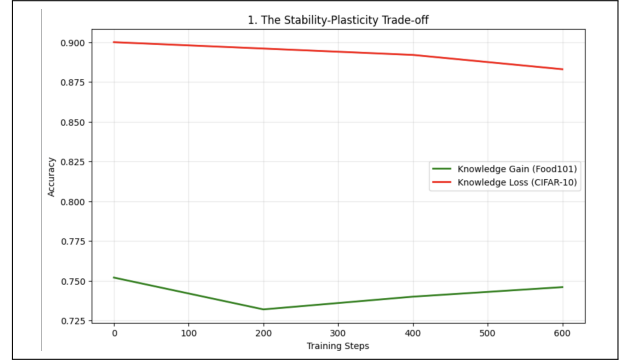


Figure 1: The Stability-Plasticity Trade-off. Green line indicates Food101 accuracy. Red line indicates CIFAR-10 accuracy.

progresses, the Food101 curve actually initially decreases while the CIFAR-10 curve exhibits a clear downward trend. Then at around 200 steps the CIFAR-10 curve increases and Food101 has a sharp drop at 400 epochs. The gap between the two curves narrows over time, which indicates that specializing on Food101 degrades performance on the held-out general-domain task.

5.2 Geometric Characterization of Drift

To understand how this performance trade-off manifests in representation space, we analyze geometric drift on CIFAR-10 embeddings over training. Figure 2 plots two metrics as a function of checkpoint step: cosine-based centroid displacement and Fréchet Distance relative to the baseline embeddings.

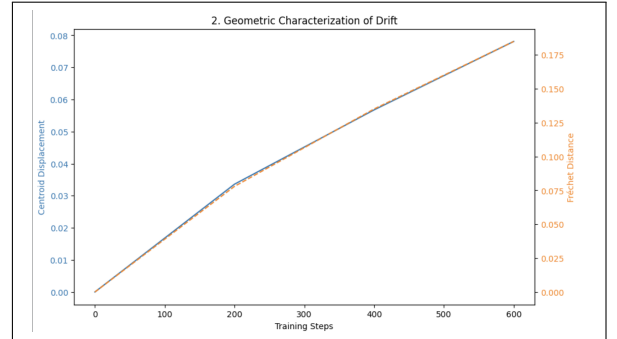


Figure 2: Geometric Characterization of Representation Drift. Blue solid line: Centroid Displacement (Cosine Drift). Orange dashed line: Fréchet Distance.

This plot shows that the geometric misalignment, or Representation Drift, of the model’s fundamental knowledge is not random but follows a continuous, systematic path during training. The near-identical movement of the Centroid Displacement (measuring the average shift of concepts like ‘cat’ or ‘dog’) and the Fréchet Distance (measuring how scattered those concepts become) proves that the entire learned space is undergoing a coordinated structural warp as the model learns the new domain. This consistent, measurable

drift confirms that these geometric metrics are reliable proxies for monitoring the silent decay of the model's general capabilities.

5.3 Drift vs. Forgetting Correlation

We next examine how closely geometric drift tracks downstream performance loss on CIFAR-10. Figure 3 overlays the CIFAR-10 accuracy drop (relative to the baseline) and the CIFAR-10 Fréchet Distance across checkpoints on a shared time axis.

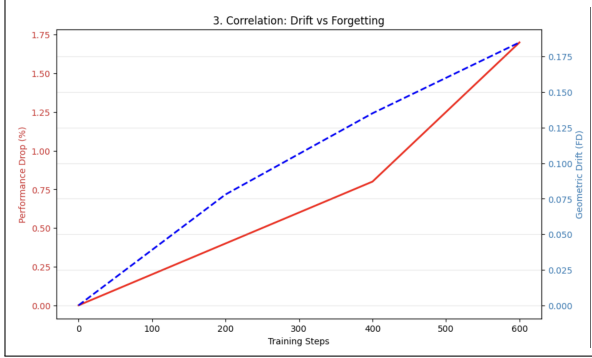


Figure 3: Geometric Drift vs. Downstream Forgetting. Red solid line: Performance drop on CIFAR-10 (percentage points). Blue dashed line: Fréchet Distance on CIFAR-10.

This plot shows the direct, causal link between the model's internal changes (Drift) and its performance failures (Forgetting). The red line (Performance Drop) represents how much general knowledge the model has lost, while the blue dashed line (Geometric Drift) represents the quantifiable physical warping of the model's internal knowledge space. This proves that the geometrical integrity of the embeddings is the mechanism driving accuracy. As the model warps its internal map to learn Food101, the forgetting occurs in a predictable, synchronized manner.

5.4 Class-Level Degradation Analysis

While aggregate metrics highlight global trends, they do not reveal which classes are most affected. To study per-class behavior, we compute a confusion matrix for CIFAR-10 at the final checkpoint and compare it to the true labels used at baseline. Figure 4 shows the normalized confusion matrix constructed from prototype-based predictions on the fixed CIFAR-10 evaluation subset.

This heatmap visually diagnoses how the model is starting to fail by the end of the training process. The bright colors on the main diagonal show the model is mostly correct, but the faint purple squares appearing elsewhere (off the diagonal) show where the model is getting confused. Specifically, the drift causes semantic blurring: the model struggles most with distinguishing between very similar general concepts like 'Cat' vs 'Dog' or 'Bird' vs 'Frog'. This proves that the fine-tuning process, by forcing the model to focus on new food textures, inadvertently destroys the crucial, subtle features it needs to tell similar animals apart.

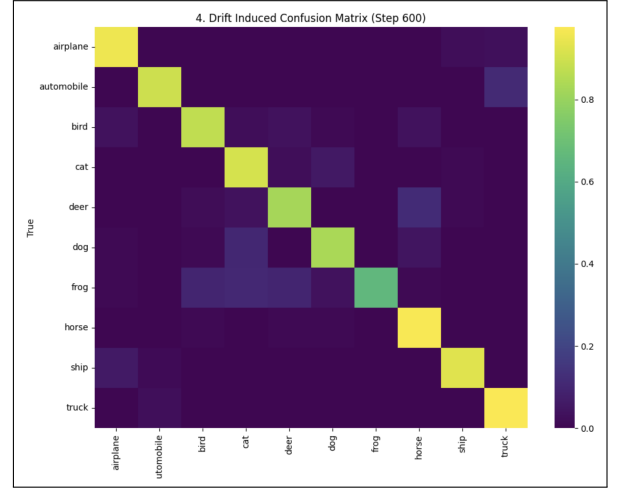


Figure 4: Drift Induced Confusion Matrix (Final Checkpoint). Normalized confusion matrix for CIFAR-10 using prototype-based predictions after fine-tuning.

5.5 Visualizing Latent Space Topology

Finally, we use t-SNE to visualize how the latent geometry of selected CIFAR-10 classes changes between the baseline and the final checkpoint. Figure 5 depicts two-dimensional projections for three representative classes (Airplane, Auto, Cat), with circles denoting baseline embeddings and crosses denoting embeddings after fine-tuning.

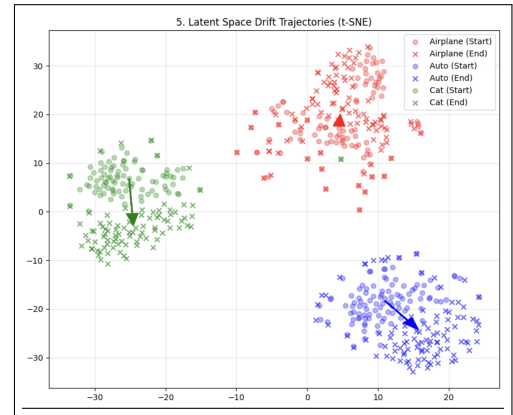


Figure 5: t-SNE Trajectories. Start (circles) and end (crosses) positions for selected CIFAR-10 classes, with arrows indicating the shift in class centroids.

Within each class, the cluster of points maintains a coherent shape, but the centroid shifts in a consistent direction between the start and end of training, as indicated by the arrows. This behavior suggests that representation drift is not purely random noise: the model applies a structured transformation that relocates whole semantic clusters in the embedding space while preserving some

within-class cohesion. Together with the global metrics and confusion analysis, this visualization reinforces the view that LoRA-based specialization produces a systematic shearing of the latent space that underlies the observed forgetting on CIFAR-10.

6 Conclusion

In this project, we implemented and evaluated a complete framework for detecting and characterizing representation drift in a fine-tuned vision-language model. By adapting CLIP ViT-B/32 to the Food101 domain using LoRA adapters on the vision tower and monitoring CIFAR-10 as a held-out proxy for general knowledge, we obtained a controlled setting in which learning and forgetting could be measured jointly across training checkpoints. Our experiments combined prototype-based accuracy with cosine centroid displacement, Fréchet Distance, confusion matrices, and t-SNE visualizations to study how performance changes relate to geometric shifts in the embedding space.

The empirical results support three main observations. First, specialization on Food101 improves in-domain accuracy while coinciding with a consistent drop in CIFAR-10 performance, illustrating a clear stability-plasticity trade-off in this dual-dataset configuration. Second, representation drift is measurable in practice: both cosine drift and Fréchet Distance on CIFAR-10 increase over training and exhibit trajectories that qualitatively track the observed accuracy drop, suggesting that geometric metrics provide a useful signal about emerging degradation on the held-out domain. Third, the combination of confusion analysis and t-SNE projections indicates that the drift is structured rather than purely random, with semantic clusters moving in a coherent fashion and certain class confusions becoming more pronounced after fine-tuning.

Future Work: Our measurements raise several directions for extension. One avenue is to move from passive monitoring to active control by designing training-time criteria that automatically halt or regularize fine-tuning when geometric drift exceeds a task-dependent threshold. Another promising direction is to explore gradient-projection or subspace-constrained update rules that explicitly protect the covariance structure of embeddings on the forget set while still allowing LoRA adapters to adapt in orthogonal directions. Finally, applying the same framework to additional domains and architectures, and comparing alternative PEFT methods, would help clarify how general these drift patterns are and how best to balance specialization with long-term stability.

7 References

References

- [1] Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML.
- [2] Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.
- [3] Wortsman, M., et al. (2022). Robust fine-tuning of zero-shot models. CVPR.
- [4] Bossard, L., et al. (2014). Food-101 – Mining Discriminative Components with Random Forests. ECCV.
- [5] Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images.

8 Appendix: Implementation Code

```

1  '''
2  CS 5787 Deep Learning Final Project
3  Team: Sahil Mhatre, Ali Hasan, Corey Chen
4  '''
5  # Used ChatGPT to help with code readability, structuring, saving to google drive, plot generation, training metrics,
6  # along with setup of models and loading data
7  import sys
8  import os
9  import subprocess
10 from pathlib import Path
11 import shutil
12 import random
13 from datetime import datetime
14 import pandas as pd
15 import numpy as np
16 import matplotlib.pyplot as plt
17 import seaborn as sns
18 from sklearn.manifold import TSNE
19 from sklearn.metrics import confusion_matrix
20 from scipy.linalg import sqrtm
21 from tqdm import tqdm
22 import torch
23 import torch.nn as nn
24 from torch.optim import AdamW
25 from torch.utils.data import DataLoader
26
27 # PART 0: SETUP & REPRODUCIBILITY
28 def set_seed(seed=42):
29     random.seed(seed)
30     np.random.seed(seed)
31     torch.manual_seed(seed)
32     torch.cuda.manual_seed_all(seed)
33
34 set_seed(42)
35
36 if not torch.cuda.is_available():
37     sys.exit("NO GPU")
38 print(f"{torch.cuda.get_device_name(0)}")
39
40 from google.colab import drive
41 drive.mount('/content/drive', force_remount=True)
42 project_dir = Path('/content/drive/MyDrive/drift_analysis_final_merged')
43 project_dir.mkdir(exist_ok=True)
44 os.chdir(project_dir)
45 for d in ['embeddings', 'checkpoints', 'results', 'logs', 'plots']:
46     Path(d).mkdir(exist_ok=True)
47
48 subprocess.run(['pip', 'install', '-q', 'transformers', 'accelerate', 'peft', 'datasets', 'scikit-learn', 'scipy', 'umap',
49               '-learn', 'tqdm', 'seaborn'], check=True)
50
51 # PART 1: DATASET PREPARATION
52 from datasets import load_dataset
53 from transformers import CLIPModel, CLIPProcessor, CLIPTokenizer
54 from peft import LoraConfig, get_peft_model
55
56 # A. CIFAR-10 (Forget Set / Stability)
57 cifar = load_dataset("cifar10", split="test")
58 cifar_classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
59 cifar_eval = cifar.shuffle(seed=42).select(range(1000))
60
61 def prepare_cifar(example):
62     example['caption'] = f"a photo of a {cifar_classes[example['label']]}"
63     example['image'] = example['img']
64     return example
65 cifar_eval = cifar_eval.map(prepare_cifar)

```

```

66 # B. Food101 (Learn Set / Plasticity)
67 food_dataset = load_dataset("food101", split="train")
68 food_classes = food_dataset.features['label'].names
69 food_split = food_dataset.train_test_split(test_size=1000, seed=42)
70 food_train = food_split['train'].select(range(5000))
71 food_eval = food_split['test']
72
73 print(f"Training Data: {len(food_train)} (Food101)")
74 print(f"Fixed Eval: CIFAR ({len(cifar_eval)}, Food ({len(food_eval)}))")
75
76 # PART 2: MODEL CONFIGURATION
77 device = torch.device("cuda")
78 model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
79 processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
80 tokenizer = CLIPTokenizer.from_pretrained("openai/clip-vit-base-patch32")
81
82 # LoRA Config
83 lora_config = LoraConfig(
84     r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"],
85     lora_dropout=0.1, bias="none"
86 )
87 model.vision_model = get_peft_model(model.vision_model, lora_config)
88 model.vision_model.print_trainable_parameters()
89
90 # PART 3: UTILITIES
91 def get_class_features(class_names):
92     texts = [f"a photo of a {c}" for c in class_names]
93     inputs = tokenizer(texts, return_tensors="pt", padding=True).to(device)
94     with torch.no_grad():
95         emb = model.get_text_features(**inputs)
96         emb = emb / emb.norm(dim=-1, keepdim=True)
97     return emb.cpu().numpy()
98
99 cifar_prototypes = get_class_features(cifar_classes)
100 food_prototypes = get_class_features(food_classes)
101
102 def eval_collate_fn(batch):
103     images = [item['image'].convert('RGB') for item in batch]
104     labels = [item['label'] for item in batch]
105     captions = []
106     for item in batch:
107         if 'caption' in item:
108             captions.append(item['caption'])
109         else:
110             captions.append(f"a photo of {food_classes[item['label']]}")
111     return processor(text=captions, images=images, return_tensors="pt", padding=True), labels
112
113 def extract_embeddings_batched(model, dataset, desc="Extracting"):
114     model.eval()
115     img_embs, all_labels = [], []
116     loader = DataLoader(dataset, batch_size=64, shuffle=False, collate_fn=eval_collate_fn, num_workers=2)
117
118     for inputs, labels in tqdm(loader, desc=desc, leave=False):
119         inputs = {k: v.to(device) for k, v in inputs.items()}
120         with torch.no_grad():
121             out = model(**inputs)
122             i_norm = out.image_embeds / out.image_embeds.norm(dim=-1, keepdim=True)
123             img_embs.append(i_norm.cpu().numpy())
124             all_labels.extend(labels)
125
126     return {
127         'image_embs': np.vstack(img_embs),
128         'labels': np.array(all_labels)
129     }
130
131 # Baseline Extraction
132 base_cifar = extract_embeddings_batched(model, cifar_eval, desc="Baseline CIFAR")
133 base_food = extract_embeddings_batched(model, food_eval, desc="Baseline Food")
134 np.savez('embeddings/baseline_cifar.npz', **base_cifar)

```



```

135 np.savez('embeddings/baseline_food.npz', **base_food)
136
137 # PART 4: TRAINING LOOP (With Dual Checkpointing)
138 def train_collate_fn(batch):
139     images = [item['image'].convert('RGB') for item in batch]
140     labels = [item['label'] for item in batch]
141     captions = [f"a photo of {food_classes[label]}" for label in labels]
142     return processor(text=captions, images=images, return_tensors="pt", padding=True)
143
144 train_loader = DataLoader(food_train, batch_size=32, shuffle=True, collate_fn=train_collate_fn, num_workers=2)
145 optimizer = AdamW(model.vision_model.parameters(), lr=5e-5, weight_decay=0.01)
146
147 TOTAL_EPOCHS = 5
148 CHECKPOINT_FREQ = 200
149 global_step = 0
150 checkpoint_steps = [0]
151
152 # Save Step 0
153 np.savez(f'embeddings/step_0_cifar.npz', **base_cifar)
154 np.savez(f'embeddings/step_0_food.npz', **base_food)
155
156 print(f"{TOTAL_EPOCHS} Epochs.")
157
158 for epoch in range(TOTAL_EPOCHS):
159     model.train()
160     pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}")
161
162     for batch in pbar:
163         batch = {k: v.to(device) for k, v in batch.items()}
164
165         # Standard CLIP Loss
166         out = model(**batch)
167         i_emb = out.image_embeds / out.image_embeds.norm(dim=-1, keepdim=True)
168         t_emb = out.text_embeds / out.text_embeds.norm(dim=-1, keepdim=True)
169         logits = (i_emb @ t_emb.T) / 0.07
170         labels = torch.arange(len(logits), device=device)
171         loss = (nn.functional.cross_entropy(logits, labels) + nn.functional.cross_entropy(logits.T, labels)) / 2
172
173         optimizer.zero_grad()
174         loss.backward()
175         optimizer.step()
176
177         global_step += 1
178         pbar.set_postfix({'loss': f"{loss.item():.4f}"})
179
180         if global_step % CHECKPOINT_FREQ == 0:
181             # EVAL ON BOTH CIFAR AND FOOD
182             ckpt_cifar = extract_embeddings_batched(model, cifar_eval, desc="Eval CIFAR")
183             ckpt_food = extract_embeddings_batched(model, food_eval, desc="Eval Food")
184
185             np.savez(f'embeddings/step_{global_step}_cifar.npz', **ckpt_cifar)
186             np.savez(f'embeddings/step_{global_step}_food.npz', **ckpt_food)
187             checkpoint_steps.append(global_step)
188             model.train()
189
190 print("Training Complete.")
191
192 # PART 5: METRIC CALCULATION
193 def calculate_frechet(mu1, sigma1, mu2, sigma2):
194     ssdiff = np.sum((mu1 - mu2)**2.0)
195     covmean = sqrtm(sigma1.dot(sigma2))
196     if np.iscomplexobj(covmean): covmean = covmean.real
197     return ssdiff + np.trace(sigma1 + sigma2 - 2.0 * covmean)
198
199 def compute_metrics(baseline_data, current_data, prototypes):
200     # 1. Accuracy
201     logits = current_data['image_embeds'] @ prototypes.T
202     preds = np.argmax(logits, axis=1)
203     acc = (preds == current_data['labels']).mean()

```

```

204
205 # 2. Geometry: Cosine Drift (Centroid Shift)
206 base_mu = np.mean(baseline_data['image_embeds'], axis=0)
207 curr_mu = np.mean(current_data['image_embeds'], axis=0)
208 drift_cos = 1 - np.dot(base_mu/np.linalg.norm(base_mu), curr_mu/np.linalg.norm(curr_mu))
209
210 # 3. Geometry: Frchet Distance (Distribution Shift)
211 base_sigma = np.cov(baseline_data['image_embeds'], rowvar=False)
212 curr_sigma = np.cov(current_data['image_embeds'], rowvar=False)
213 fd = calculate_frechet(base_mu, base_sigma, curr_mu, curr_sigma)
214
215 return acc, drift_cos, fd
216
217 results = []
218 # Pre-compute baselines for accuracy tracking
219 base_c_acc, _, _ = compute_metrics(base_cifar, base_cifar, cifar_prototypes)
220 base_f_acc, _, _ = compute_metrics(base_food, base_food, food_prototypes)
221
222 print(f"\nBaseline Acc: CIFAR={base_c_acc:.4f}, Food={base_f_acc:.4f}")
223
224 # --- ROBUSTNESS FIX: Copy to local ---
225 local_emb_dir = Path('/content/local_embeddings')
226 local_emb_dir.mkdir(exist_ok=True, parents=True)
227 print("Copying embeddings to local runtime to prevent Drive timeouts...")
228
229 def safe_load_npz(path):
230     with np.load(path) as data:
231         return {k: data[k] for k in data.files}
232
233 try:
234     # Copy files
235     for f in Path('embeddings').glob('*.npz'):
236         shutil.copy(f, local_emb_dir)
237 except Exception as e:
238     print(f"Note: Copy warning (Drive might be unstable): {e}")
239 # -----
240
241 for step in tqdm(checkpoint_steps, desc="Computing Metrics"):
242     # Load from local if possible, else fall back (safely)
243     c_path = local_emb_dir / f'step_{step}_cifar.npz'
244     f_path = local_emb_dir / f'step_{step}_food.npz'
245     if not c_path.exists(): c_path = f'embeddings/step_{step}_cifar.npz'
246     if not f_path.exists(): f_path = f'embeddings/step_{step}_food.npz'
247
248     curr_cifar = safe_load_npz(c_path)
249     curr_food = safe_load_npz(f_path)
250
251     c_acc, c_drift, c_fd = compute_metrics(base_cifar, curr_cifar, cifar_prototypes)
252     f_acc, _, _ = compute_metrics(base_food, curr_food, food_prototypes)
253
254     results.append({
255         'step': step,
256         'cifar_acc': c_acc, 'cifar_drift': c_drift, 'cifar_fd': c_fd,
257         'food_acc': f_acc,
258         'acc_drop': base_c_acc - c_acc
259     })
260
261 df = pd.DataFrame(results)
262 df.to_csv('results/final_merged_metrics.csv', index=False)
263
264 # PART 6: GENERATING ALL 5 VISUALIZATIONS
265 # 1. Stability-Plasticity Trade-off
266 plt.figure(figsize=(10, 6))
267 plt.plot(df['step'], df['food_acc'], 'g-', linewidth=2, label='Knowledge Gain (Food101)')
268 plt.plot(df['step'], df['cifar_acc'], 'r-', linewidth=2, label='Knowledge Loss (CIFAR-10)')
269 plt.xlabel('Training Steps')
270 plt.ylabel('Accuracy')
271 plt.title('1. The Stability-Plasticity Trade-off')
272 plt.legend()

```

```

273 plt.grid(True, alpha=0.3)
274 plt.savefig('plots/fig1_tradeoff.png')
275
276 # 2. Geometric Characterization
277 fig, ax1 = plt.subplots(figsize=(10, 6))
278 ax1.set_xlabel('Training Steps')
279 ax1.set_ylabel('Centroid Displacement', color='tab:blue')
280 ax1.plot(df['step'], df['cifar_drift'], color='tab:blue', label='Cosine Drift')
281 ax1.tick_params(axis='y', labelcolor='tab:blue')
282 ax2 = ax1.twinx()
283 ax2.set_ylabel('Fr chet Distance', color='tab:orange')
284 ax2.plot(df['step'], df['cifar_fd'], color='tab:orange', linestyle='--', label='Fr chet Distance')
285 ax2.tick_params(axis='y', labelcolor='tab:orange')
286 plt.title('2. Geometric Characterization of Drift')
287 plt.savefig('plots/fig2_geometry.png')
288
289 # 3. Drift Correlation
290 fig, ax1 = plt.subplots(figsize=(10, 6))
291 ax1.set_xlabel('Training Steps')
292 ax1.set_ylabel('Performance Drop (%)', color='tab:red')
293 ax1.plot(df['step'], df['acc_drop']*100, 'r-', label='Accuracy Drop', linewidth=2)
294 ax1.tick_params(axis='y', labelcolor='tab:red')
295 ax2 = ax1.twinx()
296 ax2.set_ylabel('Geometric Drift (FD)', color='tab:blue')
297 ax2.plot(df['step'], df['cifar_fd'], 'b--', label='Fr chet Distance', linewidth=2)
298 ax2.tick_params(axis='y', labelcolor='tab:blue')
299 plt.title('3. Correlation: Drift vs Forgetting')
300 plt.grid(True, alpha=0.3)
301 plt.savefig('plots/fig3_correlation.png')
302
303 # 4. Confusion Matrix (Final Step)
304 final_c_path = local_emb_dir / f'step_{checkpoint_steps[-1]}_cifar.npz'
305 if not final_c_path.exists(): final_c_path = f'embeddings/step_{checkpoint_steps[-1]}_cifar.npz'
306 final_cifar = safe_load_npz(final_c_path)
307
308 logits_final = final_cifar['image_embeds'] @ cifar_prototypes.T
309 preds_final = np.argmax(logits_final, axis=1)
310 cm = confusion_matrix(base_cifar['labels'], preds_final, normalize='true')
311 plt.figure(figsize=(10, 8))
312 sns.heatmap(cm, annot=False, cmap='viridis', xticklabels=cifar_classes, yticklabels=cifar_classes)
313 plt.title(f'4. Drift Induced Confusion Matrix (Step {checkpoint_steps[-1]})')
314 plt.xlabel('Predicted')
315 plt.ylabel('True')
316 plt.tight_layout()
317 plt.savefig('plots/fig4_confusion.png')
318
319 # 5. t-SNE Trajectory
320 target_classes = [0, 1, 3]
321 indices = [i for i, l in enumerate(base_cifar['labels']) if l in target_classes]
322 subset_base = base_cifar['image_embeds'][indices]
323 subset_final = final_cifar['image_embeds'][indices]
324 subset_labels = base_cifar['labels'][indices]
325 combined = np.vstack([subset_base, subset_final])
326 tsne = TSNE(n_components=2, random_state=42, perplexity=30)
327 projections = tsne.fit_transform(combined)
328 mid = len(subset_base)
329 base_proj = projections[:mid]
330 final_proj = projections[mid:]
331
332 plt.figure(figsize=(10, 8))
333 colors = ['red', 'blue', 'green']
334 labels = ['Airplane', 'Auto', 'Cat']
335 for i, cls in enumerate(target_classes):
336     mask = subset_labels == cls
337     plt.scatter(base_proj[mask, 0], base_proj[mask, 1], c=colors[i], alpha=0.3, label=f'{labels[i]} (Start)', marker='o')
338     plt.scatter(final_proj[mask, 0], final_proj[mask, 1], c=colors[i], alpha=0.6, label=f'{labels[i]} (End)', marker='x')
339 cent_start = np.mean(base_proj[mask], axis=0)

```

```
340     cent_end = np.mean(final_proj[mask], axis=0)
341     plt.arrow(cent_start[0], cent_start[1], cent_end[0]-cent_start[0], cent_end[1]-cent_start[1],
342             head_width=1.5, color=colors[i], length_includes_head=True, linewidth=2)
343 plt.title('5. Latent Space Drift Trajectories (t-SNE)')
344 plt.legend()
345 plt.grid(True, alpha=0.3)
346 plt.savefig('plots/fig5_tsne.png')
347
348 # Zip Results
349 shutil.make_archive('Merged_Final_Submission', 'zip', project_dir)
```